Password Generator and Checker

Module	Programming Foundations – Retake Examination
Author	João-Manoel Roth
Programme	BSc Business Information Technology
University	FHNW – University of Applied Sciences Northwestern Switzerland
Submitted	30 June 2026

1. Rationale for the Topic

Most people reuse the same simple passwords because creating strong, unique ones manually is time-consuming. Cloud-based password managers require an internet connection and raise privacy concerns for many users.

This project provides a lightweight, fully offline console application that generates cryptographically random passwords, evaluates password strength, and saves results locally — with no third-party dependencies.

Mandatory Criterion	How This Project Satisfies It
Interactive app (console input processing)	A continuous while-loop menu lets the user choose between password generation, strength checking, and quitting. All interaction happens via the console.
Data validation (type / format checking)	prompt_int() rejects non-integer and below-minimum inputs. prompt_yes_no() only accepts y / yes / n / no. An empty-string guard prevents blank password checks.
File processing (reading and/or writing data)	save_password() automatically creates the data/ directory and appends each generated password to passwords.txt in UTF-8 encoding (append mode).

2. Project Goals and Features

2.1 User Stories

#	Title	Description
US 1	**Interactivity**	As a user I want to navigate a text-based console menu so I can choose between password generation and strength checking.
US 2	**Data Validation**	As a user I want the system to validate my inputs immediately so the program remains stable even when I enter invalid data.
US 3	**File Processing**	As a user I want generated passwords to be saved in a local text file so I can access them again later.
US 4	**Functionality**	As a user I want to enter any password and immediately learn whether it is rated Weak, Medium, or Strong based on predefined criteria.

2.2 Strength Classification Criteria

Rating	Length	Required Character Categories
Strong	≥ 12 chars	All four: uppercase, lowercase, digit, special character
Medium	≥ 8 chars	At least three of the four categories
Weak	< 8 chars	Fewer than three categories present (including empty string)

3. Project Management

3.1 Timeline

Date	Milestone
22 May 2026	Set up repository structure (src, data, tests, docs, .gitignore, README).
29 May 2026	Implement generate_password() and all input validation functions.
5 June 2026	Implement save_password(), check_password_strength(), and main() loop.
12 June 2026	Extensive testing, fix indentation, add empty-input guard, run test suite.
30 June 2026	Code freeze, create PDF presentation, submit ZIP file on Moodle.

3.2 Highlights

- Clean separation of concerns: each function has a single, well-defined responsibility.
- No external dependencies — the entire application uses only Python's standard library.
- Guaranteed character-category coverage: the generator always seeds one character from each selected category before filling the remainder randomly.
- Password length is validated with both a minimum (8) and a maximum (64) to prevent unrealistic inputs and guide the user clearly.
- Weak password results include an actionable tip to help the user improve their password.
- Fully automated test suite (37 tests, 4 sections) covering boundary values, edge cases, file handling, and max_value validation — all 37 tests passed.
- Public GitHub repository with a clear commit history and a descriptive README.

3.3 Challenges and How They Were Solved

Mixed indentation (tabs vs. spaces)

The initial version mixed 2-space and tab indentation across functions. Caught during code review and corrected to uniform 4-space indentation throughout, following PEP 8.

Empty password input in Option 2

Pressing Enter without input caused check_password_strength() to silently return 'Weak' on an empty string. An explicit guard (if not pw) was added to print 'No password entered.' and return to the menu before the strength check runs.

File path resolution

Hard-coded relative paths broke when the script was run from a different working directory. Fixed using os.path.abspath + os.path.dirname(__file__) to resolve DATA_DIR relative to the script's own location.

Guaranteed character-category coverage

A naive random.choices() call occasionally produced passwords missing digits or uppercase letters. Solved by seeding the list with one guaranteed character from each required category before filling the remainder randomly.

4. Technical Implementation

4.1 Project Structure

Path	Purpose
`src/main.py`	All application source code (single-file design)
`data/passwords.txt`	Auto-created on first run; stores generated passwords in append mode
`tests/`	Directory for test scripts
`docs/`	Directory for documentation and this presentation
`.gitignore`	Excludes __pycache__ and generated files from version control
`README.md`	Short project description on GitHub

4.2 Standard Library Modules Used

- `random` – Random character selection for password generation
- `string` – Predefined character sets: ascii_lowercase, ascii_uppercase, digits, punctuation
- `os` – File system operations: create directories, join paths, resolve absolute paths
- `sys` – Interpreter access: sys.exit() for a clean program exit with a return code

4.3 Key Functions

`generate_password(length, include_special)`

Generates a random password of the requested length. Always includes at least one lowercase letter, one uppercase letter, and one digit. Adds one special character if requested. The list is shuffled to avoid predictable patterns. Raises ValueError if length < 8.

`check_password_strength(password)`

Classifies a password as Strong (≥ 12 chars, all four categories), Medium (≥ 8 chars, at least three categories), or Weak (anything else). Handles empty strings correctly.

`save_password(password)`

Creates data/ if missing, then appends the password to passwords.txt with a newline (UTF-8 encoding, append mode).

`prompt_int(prompt, min_value, max_value)`

Reads an integer from stdin. Repeats on ValueError and rejects values below the optional minimum or above the optional maximum.

`prompt_yes_no(prompt)`

Accepts y / yes / n / no (case-insensitive). Repeats on any other input.

`main()`

Entry point. Runs the menu loop, dispatches to the correct function, and handles KeyboardInterrupt and EOFError for graceful termination.

5. Data Validation

Every user input is validated before processing. The program never crashes on invalid input — it always shows a clear error message and re-prompts:

Input Scenario	Program Response
Letter instead of number (e.g. 'abc')	ValueError caught → 'Invalid input — please enter an integer.'
Number below minimum (e.g. length = 7)	'Input must be at least 8.'
Number above maximum (e.g. length = 100)	'Input must be at most 64.'
Invalid yes/no answer (e.g. 'maybe')	'Please answer "y" or "n".'
Empty password in Option 2 (Enter only)	'No password entered.' — strength check is skipped
Weak password in Option 2	'Strength: Weak' followed by a tip suggesting how to improve the password
Invalid menu option (e.g. '5')	'Invalid selection — please choose 1, 2 or 3.'
Ctrl+C or EOF	Graceful exit: 'Program terminated.' — return code 0

6. File Processing

The save_password() function handles all file I/O automatically:

- The `data/` directory is created on first use with `os.makedirs(DATA_DIR, exist_ok=True)` — no error if it already exists.
- The file is opened in **append mode ('a')** so existing entries are never overwritten.
- Each password is written with an explicit newline character.
- Encoding is explicitly set to **UTF-8** to handle special characters correctly.
- After saving, the full file path is printed to the console so the user knows where the file is.

7. Test Results

A dedicated automated test suite (tests/test_suite.py) was written and executed. All 37 tests passed:

Test Section	Tests	Result
generate_password() — length accuracy, character categories, randomness, error cases (length 7, 0, negative)	16	✓ All passed
check_password_strength() — Strong / Medium / Weak classification, all boundary values (length 7, 8, 11, 12)	15	✓ All passed
save_password() / file handling — auto-create directory, correct content, append persistence	3	✓ All passed
prompt_int() — max_value=64 rejects values above limit, rejection message shown, min_value rejects values below limit	3	✓ All passed
Total	**37**	**✓ 37 / 37**

Manual Testing

In addition to the automated suite, the application was tested manually in the terminal. All menu paths, invalid inputs, and edge cases were verified. Generated passwords were confirmed to appear correctly in data/passwords.txt. The program behaved stably in all tested scenarios.

8. GitHub Repository

The complete source code is publicly available at:

`https://github.com/joaomanoel44/Password-Generator-and-Checker`

Key commits:

- Initial commit: project structure created (src, data, tests, docs, .gitignore)
- feat: complete source code added to src/main.py
- fix: normalise indentation to 4 spaces and add empty input guard in strength checker
- chore: remove __pycache__ from version control
- feat: add maximum password length of 64 to input validation
- ux: inform user of valid password length range before prompting
- ux: add improvement tip when password is rated Weak
- docs: add project README with features, usage and structure
- docs: add PDF presentation to docs/

9. Declaration of Independent Work

I hereby confirm that I have completed this project work independently and without unauthorised assistance. All sources and tools used have been correctly acknowledged. This work has not been submitted in the same or a similar form to any other examination authority.

Place, Date	**Signature**
Basel, 30 June 2026	João-Manoel Roth